Compiling Facts into Applications

Samuel Lippert
Drivly Inc

Abstract

A database schema, a family of state-machine workflows, and a REST API with hypermedia navigation are usually three artifacts kept in sync by hand. They are one object. Backus's Applicative State Transition system [1] carries a *FILE* store it never structures; Codd's relations [2] supply the structure, Object-Role Modeling [4] supplies the processes, and the representation function ρ identifies them: set membership is function application. An *application* is then this object, and running it is finite reduction over a finite fact population, which is what makes it algorithmic at all. The mapping's faithfulness rests on the formalisms it instantiates, and the only informal residue is the registered platform layer, an enumerable surface that coincides with the decidability frontier. The object is self-similar: it rewrites its own schema and contains isolated copies of itself, so a tenant is a sub-store and a tenant's tenants are sub-sub-stores, and each is a full instance carrying the same guarantees. A reference implementation is available [14].

1 Foundation

Three abbreviations are overloaded terms: *AST* is Backus's Applicative State Transition [1, Sec. 14], not the compiler's syntax tree; *ORM* is Halpin's Object-Role Modeling [4], not the persistence pattern; *FP* and *FFP* are Backus's function-level systems [1, Secs. 11, 13]. The construction assumes these.

At the bottom there are two things, and they are the same thing. A set is its characteristic function, so for a population P the membership question $x \in P$ and the application $P\,x$ are one act: **membership is application**. The fact base is therefore not data beside a program; it *is* the function space, and Backus's representation function ρ [1, Sec. 13.3], which maps an object to the function it denotes, is the reflection exposing a fact as the function it already is. A fact type $\langle CONS, s_1, \ldots, s_n \rangle$ [1, Sec. 11.2.4] is a constructor of roles; populate the roles and the resulting fact is resolved by looking up its type:

$$(\rho\langle x_1, \ldots, x_n \rangle) : y = (\rho\, x_1) : \langle \langle x_1, \ldots, x_n \rangle, y \rangle. \qquad (1)$$

This is metacomposition [1, Sec. 13.3.2]. It is the only mechanism in the paper, and every later view (a constraint, a transition, a link, an API response) is an instance of it. The combining forms it composes are also Backus's [1, Sec. 11.2.4]: composition $f \circ g$, the selectors, the equality test *eq*, the constant $\bar{x}$, and the empty sequence φ.

Backus builds an AST system from three parts [1, Sec. 14.3]: an applicative subsystem (an FFP system), a state D that is its set of definitions, and transition rules carrying inputs to outputs and D to D'. The state is a sequence of cells with fetch $\uparrow n : D$ and store $\downarrow n : \langle x, D \rangle$ [1, Secs. 13.3.4, 14.3]. One cell is named *FILE* and "the system maintains" its contents, but Backus declines to structure them: "we have not said how the system's file, queries or updates are structured" [1, Sec. 14.4.4]. AREST fills exactly that hole.

Definition 1 (AREST)**.** An *Applicative REpresentational State Transfer* system is a Backus AST system [1, Sec. 14] whose *FILE* cell contains a population P (Definition 3) of a schema S (Definition 2), and whose definition cells *DEFS* hold the FFP objects [1, Sec. 13] compiled from FORML 2 readings together with those registered by the runtime. REST operations are applications of the system function (2).

A small declaration fixes the picture. In FORML 2 [5], each reading is one elementary fact:

```
Order(.OrderId) is an entity type.
Customer(.Name) is an entity type.
Order is placed by Customer.
Each Order is placed by exactly one
  Customer.
Customer ships Order.
State Machine Definition 'Order' is for
  Noun 'Order'.
Status 'In Cart' is initial in
  State Machine Definition 'Order'.
Transition 'place' is from
  Status 'In Cart'.
Transition 'place' is to
  Status 'Placed'.
Transition 'place' is triggered by
  Fact Type 'Customer places Order'.
Transition 'ship' is from
  Status 'Placed'.
Transition 'ship' is to
  Status 'Shipped'.
Transition 'ship' is triggered by
  Fact Type 'Customer ships Order'.
```

Each reading is atomic, and each maps to a column: the entity references to keys, the binary fact types to for-

eign keys, the uniqueness reading to a restriction, and the transition facts to the machine's rows. The decomposition into atomic facts *is* the mapping to the relational schema, and together they compile to a table, a foreign key, a uniqueness restriction, and a transition that fires when its trigger fact enters P. A `POST /orders` request creates an Order and returns its status with the actions available from it; following the `place` action advances the machine to `Placed`, after which the representation offers `ship` and no longer `place`.

Finite, therefore algorithmic. The population is a finite set. This is not a modeling convenience but the precondition of computation: an algorithm is finite, so whatever an application computes lives in the finite fragment. Finiteness is meant per operation. Each system step terminates over the current finite P, while the event stream is unbounded over time, so the object stays algorithmic at every point. Finiteness earns the guarantees the rest of the paper reads off: evaluation is finite model-checking, so it is decidable; the reflection of (1) is consistent without Scott's domains, because the function space over a finite set is itself a finite set; and derivation reaches a least fixed point in finitely many steps (Lemma 1). The place where this stops, namely the arbitrary registered functions of Definition 9 whose termination is in general undecidable, is the only point at which general computation re-enters, and Corollary 5 makes it an enumerable fact set.

2 ORM over Backus's AST

We fix the objects the rest of the paper quantifies over by reading them off ORM 2 as NORMA implements it [4, 13].

Definition 2 (Schema)**.** A *vocabulary* is a set of object types, each a *value type* (self-identifying, typed by a data type) or an *entity type* (identified by a preferred uniqueness constraint over one role, simple, or several, compound), together with a set F of fact types, each $arity(f)$ roles played by object types. An entity's identifier is *auto-generated* iff its identifying data type is auto-generating (counter, UUID, timestamp, or surrogate) and *supplied* otherwise. *Val* is the union of the value-type domains and the entity identifiers. A *schema* S comprises F, a constraint set C_S (uniqueness, mandatory, frequency, ring, value-comparison, subset, equality, exclusion, value, and cardinality constraints, each alethic or deontic [13]), a derivation-rule set R_S (Definition 7), a set of state machines, and the reference schemes. S is the compiled content of *DEFS*.

Definition 3 (Population)**.** A *population* of S is a finite set

$$P \subseteq \bigcup_{f \in F} \{f\} \times Val^{arity(f)},$$

with finite *active domain* $adom(P)$, the set of values occurring in P. A ground fact g is *true* in P if $g \in P$, *false* if its paired negation fact is in P, and *unknown* otherwise; under the closed-world assumption on a noun, unknown collapses to false. Membership is the characteristic function of P, so $g \in P$ and $P\,g$ are one act.

Entity and fact types populate *FILE*, while constraints and derivations are relational restrictions and compositions over P, and the retrieval operators are Codd's adequate θ_1: projection, natural join, tie, and restriction [2, Sec. 2.2]. We write restriction as $Filter(p) : X$, the elements of a relation X satisfying a predicate p; this is Codd's θ-restriction [3, Sec. 2.3.5], with constants entering as literal relations. The equality patterns used below, compositions of *eq* with selectors and constants, reduce to the restriction of θ_1, while value ranges take the inequality θ. Indeed every constraint of Definition 2 compiles to a restriction whose predicate falls in one of two families, a cardinality count against declared bounds or a membership test against a target population (the comparison constraints taking the inequality θ), with polarity the only parameter separating symmetric from asymmetric and subset from exclusion. ORM elementary facts are in fifth normal form by construction [4], so the named relations are the cells themselves and navigation needs no separate query language. Each leg of the map is now discharged by the formalism it instantiates.

Definition 4 (Fragment)**.** R is the set of FORML 2 readings that declare the constructs of Definition 2: object-type and reference-scheme declarations, elementary fact-type readings, the listed constraints, projective derivation readings (Definition 7), and state-machine readings whose trigger is an elementary fact type. Pronoun-correlated clauses and nested objectification lie outside R and are rejected. A declared name may not contain a formal item of the grammar as a substring; the compiler refuses such a name at ingestion, and the determinism of Proposition 1 rests on this hypothesis.

Proposition 1 (Specification and executable)**.** *parse is total on R, compile $\circ$ parse is well defined, and nf = verbalize $\circ$ compile $\circ$ parse is idempotent with $nf(r) \sim r$ for the kernel equivalence $\sim$ of compile $\circ$ parse, where the executable of r is $\rho(compile(parse(r)))$.*

Proof. Each sentence of R falls in one keyword-delimited family and, under the name hypothesis of Definition 4, parses deterministically, and NORMA exhibits the round trip for all constraint kinds [13], so *parse* is total and *compile* $\circ$ *parse* is a function, injective on $R/\sim$ by construction of $\sim$. *verbalize* emits the primary reading of a compiled object [6], a sentence of R that re-parses to that object, so $nf \circ nf = nf$ and $nf(r) \sim r$. Executability is ρ [1, Sec. 13.4]. □

A state machine is itself a set of facts (a status, its transitions, and the trigger fact type of each), and advancing it is one AST step, not a second machine. Backus's transition is single: on input x the system forms $(SYSTEM : x)$, evaluates it under the current definitions, and obtains

$\mu(SYSTEM:x) = \langle o, d\rangle$ with o the output and d the next state [1, Sec. 14.3.1]; the state is frozen during evaluation [1, Sec. 14.6]. Where Backus's subsystem branches on a *KEY* cell through five clauses [1, Sec. 14.4.2], AREST routes on the addressed entity,

$$SYSTEM:x = (\rho\,(\uparrow entity(x):D))\uparrow op(x), \qquad (2)$$

so new operations and new entity types extend D without disturbing one another.

Definition 5 (Command)**.** For schema S, input I, and population P,

$$resolve_S : I \times P \to P, \qquad derive_S : P \to P,$$
$$validate_S : P \to P \times \mathcal{P}(Val^*), \qquad emit_S : P \times \mathcal{P}(Val^*) \to O,$$

where $resolve_S$ adds the entity and fact instances named by I, minting a fresh identifier exactly when the reference scheme is auto-generating; $derive_S = lfp(F_S, \cdot)$ (Definition 7); $validate_S(P) = (P, V)$ with V the violation set (Definition 6); and $emit_S$ builds the representation O. A CREATE command is

$$create(I) = emit_S \circ validate_S \circ derive_S \circ resolve_S(I, \cdot), \qquad (3)$$

committing the derived population iff V has no alethic violation, otherwise leaving D unchanged.

Definition 6 (Violation)**.** For a constraint $c \in C_S$, $V_c = (\rho\, c) : P$ is the finite set of bindings that offend c, and $V = \bigcup_{c \in C_S} V_c$. An alethic c rejects the commit when $V_c \neq \varphi$; a deontic c warns and commits. The message returned is the canonical reading of c (Proposition 1).

Definition 7 (Derivation rules)**.** A derivation rule in R_S is a role path projected onto a head: each derived role is bound to a path variable, to a calculated value (a function of finitely many bound values, an aggregate reducing a finite bag to one scalar), or to a constant [13]. A rule is *fully derived* ($*$), *derived and stored* ($**$, materialized), or *semi-derived* ($+$, also directly assertable). Heads are *projective*: no derivation rule introduces a fresh entity. The immediate-consequence operator is $F_S(P) = P \cup \{\text{heads derivable from } P \text{ by one rule}\}$. Entity introduction is confined to $resolve_S$ and to state-transition rules, each guarded by a positive event [13].

Proposition 2 (Single transition)**.** *The live state-machine step is the AST transition $\mu(SYSTEM : x) = \langle o, d\rangle$. Reconstruction $machine(s_0, E) = foldl\ transition\ s_0\ (order_\tau E)$ is a ρ-application over the event facts, not a transition.*

Proof. A triggering event is the input x and the current status a fact in D, and folding it forward is one AST step, with D frozen throughout [1, Secs. 14.3.1, 14.6]. The function *machine* is an FFP expression over the event facts, its *foldl* derived from Backus's *while* [1, Sec. 11.2.4], evaluated within such a step, so it is a ρ-application (Proposition 3) used for migration and audit; it alone orders by each event's occurrence timestamp τ, while the live step takes events in arrival order. The two orders are Halpin's valid time and transaction time [4, Sec. 13.6]: τ is a role of the event fact itself, ordinary data, so $order_\tau$ is derivable from E, whereas arrival order is the log's and is no fact of the domain. □

Lemma 1 (Finiteness)**.** *If no value-introducing rule of R_S lies on a cycle of the derivation dependency graph, then for every finite P the least fixed point $lfp(F_S, P)$ exists, equals the least model of R_S above P, and is reached by iteration in finitely many steps.*

Proof. By Definition 7 a head is projective, so F_S introduces no entity. The value-introducing rules, lying on no cycle, form a directed acyclic subgraph and produce finitely many values, each a function of finitely many inputs; let $\hat{P}$ be the resulting finite extension of $adom(P)$. Every remaining rule introduces no value, so F_S is monotone over the finite powerset of facts above $\hat{P}$. The least fixed point exists by Knaster–Tarski [8]. It equals the least model because each negated role path reads only settled facts and computes a finite-set anti-join whose value is fixed across the derivation rounds, so F_S remains monotone [7] under negation. It is reached by iteration in at most $|P_{\max}| - |P|$ steps [9], where $P_{\max}$ is that finite fact space. The one input-unbounded source of fresh values is an auto-generated reference scheme in $resolve_S$ or a state-transition rule, which mints one surrogate per guarded step, outside F_S. □

Corollary 1 (The hypothesis is a query)**.** *The hypothesis of Lemma 1 is decidable by a restriction over D itself. Which fact types a rule reads and which it derives are among the schema facts describing DEFS, and a rule's body is the impl of its definition tuple (Definition 9), so the dependency graph and the bodies are data. Value introduction is syntactic: a body applies a definition with s_{origin} = registered (the boundary (5)) or a value-constructing base operation (arithmetic, length, dynamic application); every other operation rearranges atoms already in $adom(P)$ or quoted in the rule. The check — no such rule on a dependency cycle — is finitely many reachability queries, evaluated when DEFS changes (Section 4) and refused like any alethic violation, while acyclic invention remains admissible by the proof of Lemma 1.*

Theorem 1 (Completeness of state transfer)**.** *Let S satisfy the hypothesis of Lemma 1. For input I and state D, $create(I)$ (Definition 5) yields $\langle o, D'\rangle$ with $P' = resolve_S(I, P)$, $P'' = lfp(F_S, P')$, $V = \bigcup_{c \in C_S} (\rho\, c) : P''$, and o the representation of $(P'', V, links)$; and $D' = D[FILE \mapsto P'']$ if V has no alethic violation, else $D' = D$.*

Proof. P'' exists and is finite by Lemma 1. $validate_S$ applies each $c \in C_S$ as a restriction over P'' (Definition 6), and $emit_S$ builds o from P'', V, and the links (Theorem 2). The commit rule is Definition 5. □

Theorem 2 (HATEOAS as generated projection)**.** *For an entity e, call a control* valid *iff it is an outgoing transition from $status(e)$, or its target fact type places e in one role and its remaining roles are reachable through the uniqueness structure. Then*

$$links(e) = nav(e) \cup transitions(status(e)) \qquad (4)$$

is a θ_1 expression over P and S, and it is complete*: it contains every valid control and only valid controls.*

Proof. $transitions(s)$ projects the transition facts restricted to s, and $nav(e)$ projects, for each fact type on e, the peer control (spanning unique constraint), the child controls (a non-spanning constraint makes the remaining roles children of the constrained key), and the related collections. Each is a projection over a restriction composing *eq* with selectors and constants, an equality θ-restriction [3, Sec. 2.3.5], hence within Codd's θ_1 [2, Sec. 2.2]. The validity criterion above is exactly the generation rule, so a control is emitted iff it is valid, which is completeness. □

Generating this set is the novel step. Fielding [10] fixed hypermedia as a constraint but left the controls for the server author to write by hand, and prior formal treatments model the *client*'s traversal as a machine over the representations it receives [11]. Equation (4) works the other side of the wire: the set is complete and current as a function of P and S, there are no hand-written links and no undocumented endpoints, and so a client, or an agent reading the API, has no undocumented affordance to hallucinate. A transition graph may cycle; the discipline that keeps every entity live is itself a reading, the deontic obligation that each cycle carry some exit transition, which ensures liveness without demanding structural acyclicity.

Proposition 3 (Derivability)**.** *Every value in the representation $repr(e)$, namely the selectors on e's facts, the derived facts, the constraint violations, and $links(e)$, is $(\rho\, f) : P$ for some object f.*

Proof. Selectors, derivation rules, and constraints are ρ-applications by construction, and the links are θ_1 (Theorem 2), themselves ρ-applications. No value arises outside ρ. □

Corollary 2 (No middleware)**.** *Authentication, authorization, validation, rate limiting, and transformation are each a restriction, a derivation, or a fact over P, never a layer outside ρ.*

Proof. By Proposition 3 every value is a ρ-application. Authorization is a derivation ("a User may access a Domain iff the User belongs to an Organization that manages it"), validation is $validate_S$ (Theorem 1), a rate limit is a cardinality constraint over timestamped request facts, and transformation is composition. □

The claim is semantic, not topological: a deployment may still run proxies, queues, caches, and gateways, but their application-level policy is denoted inside P rather than specified outside the object. What is usually a stack of request-intercepting services is, here, a few more readings.

Corollary 3 (Streaming)**.** *A subscription is a ρ-application that has not yet been evaluated against the current D, and an external event is a fact entering P through $\downarrow$.*

Proof. Storing into a cell with $\downarrow$ changes D, so every ρ-application over that cell re-evaluates against the new state (Proposition 3), and a subscriber is exactly such an application awaiting its next evaluation. A webhook, a queue message, and a sensor reading enter P by the same $\downarrow$, so externally fired and fact-fired updates are indistinguishable to the evaluator. □

No separate pub-sub layer, event bus, or callback registry is required.

Platform binding. A runtime registers its own functions into *DEFS*, so a fact applied to one of them yields the corresponding effect:

$$(\rho\, fact) : render \to widget,$$
$$(\rho\, fact) : httpFetch \to response,$$
$$(\rho\, fact) : upsert \to D'.$$

A browser registers rendering functions, a server registers *httpFetch* and *upsert*, and one *SYSTEM* serves browser, server, and storage by varying *DEFS* rather than the logic. Binding a user interface is then registering a *render* function, so a fact renders itself. Functions that touch external state run during $resolve_S$, and those that consume the representation run after $emit_S$, which keeps $derive_S$ pure over P.

ρ and the whole-state accessor $\rho DEFS$ are Backus's [1, Secs. 13.3, 14.3.3], and the latter grants a program access to all of D "for any purpose, including the essential one of computing the successor state." Backus represents D as an object precisely because "in AST systems we shall want to transform D by applying functions to it" [1, Sec. 13.5]; this is the licence Section 4 uses for self-modification.

Deletion needs no special operation: an entity that reaches a status with no outgoing transitions has $links(e) = \varphi$ by Theorem 2 and is excluded from query results by restriction. This is logical deletion; physical reclamation is a compaction that preserves the restricted population ρ observes.

Negation. ORM's negation is open-world and explicit. NORMA admits it inside derivation role paths, where a step, a root, or a branch may be negated [13], under the three-valued reading of Definition 3, verbalized "it is not true that," "it is known to be false that," and "it is not

known to be false that." An epistemic falsity, verbalized "it is known to be false that," enters P as an explicit negation fact and is never inferred from absence. A negated role path is an inference from absence. It evaluates to a finite-set anti-join, the mirror of Restrict, over the settled facts it reads. Lemma 1 is undisturbed because that anti-join is a single finite-set operation of fixed value over the settled population. The epistemic operators are decidable queries against the settled population, not rules that grow it. The one groundedness condition is on state-transition rules: an add or delete may not occur under negation or disjunction, and each disjunctive branch must carry a positive event [13], so a step's *effect* is positive and negation guards the condition and never the head.

3 Cells, Tenancy, and Self-Similarity

RMAP [4] assigns each entity its own cell, the 3NF row of facts depending on its key, so D is a sequence of cells and, by [1, Sec. 14.7], "a cell in one store may contain another entire store."

Definition 8 (Cell isolation)**.** At most one step may write a given cell of D at a time, and steps writing disjoint cells run concurrently.

The recalculation a write forces is bounded to the entity's cell and the role-player cells its constraints can reach, a scope the schema fixes at compile time. A constraint whose scope spans cells imposes an obligation: commits touching a shared scope must serialize. The cardinality family discharges it at the constrained key itself, a single writer per scope key that generalizes Definition 8.

Consistency and availability. The choice a network partition forces, Brewer's consistency versus availability [12], is already made per noun by the modal and world-assumption declarations. An alethic constraint over a closed-world noun with a single writer is CP: under partition the commit is refused rather than risk the invariant. A deontic constraint over an open-world noun is AP: the local step accepts optimistically, warns, and reconciles at commit, where $validate_S$ (Theorem 1) runs against the authoritative P. The posture is chosen by the schema rather than fixed globally, and it tunes with a constraint's *slack*: a cardinality constraint far from its bound needs no coordination and stays available, while one at its bound serializes at the constrained key (Definition 8). Local evaluation is optimistic feedback and never a security boundary, because acceptance is decided at commit. Among peers sharing P, an event is accepted exactly when it adds no alethic violation, so the constraint set is itself the consensus predicate, with safety from deterministic replay and liveness from the single-writer cell. Beyond two peers the shared event log needs a total order; among anonymous peers a hash-chained log supplies it, a signature carries identity, and a schema modification is admitted only under a deontic authorization constraint, so ordering, validity, identity, and permission compose from the formalism already present, each peer validating independently.

Tenancy. A tenant is this mechanism one level up: a cell whose contents is an entire store, so P is partitioned into per-tenant blocks.

Proposition 4 (Tenant isolation)**.** *For stores D_a and D_b in sibling tenant cells, no address well-formed in D_a denotes a cell of D_b. Isolation is preservation of addressability under $\uparrow$ and $\downarrow$.*

Proof. D_b is the contents of a cell that is not a cell of D_a, and $\uparrow$ and $\downarrow$ resolve only cells of their store argument [1, Secs. 13.3.4, 14.3], so no fetch or store expression formed over D_a denotes a cell of D_b. □

Isolation is therefore a property of the shape of the state rather than a check: wrong-tenant access is not forbidden but unaddressable, and the row-level perimeter usually built for this has nothing to do.

Sub-tenancy. The containment recurses. A tenant's store may itself hold tenant cells, each an entire store, to any finite depth, so the tenant tree is a tree of nested sub-stores. Every node is a full instance, since Corollary 4 holds per store, so a tenant carrying its own *DEFS* is an application builder who may in turn have tenants, while one inheriting its parent's is the ordinary data-isolated case. Visibility follows containment: a parent reaches into its children because their stores are cells in its own, and a child cannot address upward or sideways (Proposition 4). This is the isolation and visibility model of a platform-of-platforms, with no access machinery beyond the nesting.

Two recursions issue from one decision. Because Backus made D a first-class object, it can be *transformed* (the system rewrites its own schema, Section 4) and it can *contain itself* (the nested stores above). The system evolves itself and holds isolated copies of itself, and both fall out of the same move.

4 Closure and the Enumerable Boundary

Self-modification is an ordinary step: the addressed entity is *DEFS* and the operation is $compile \circ parse$, with the new objects entering through $\downarrow DEFS$.

Corollary 4 (Closure)**.** *A self-modifying step changes only DEFS and the schema facts describing it; the evaluator, parser, compiler, verbalizer, and registered functions are unchanged. After a step whose operation is $compile \circ parse$ on readings $R' \subseteq R$, Definitions 2–7, Propositions 1 and 3, Lemma 1, and Theorems 1 and 2 hold for every later step over D'.*

Proof. No statement's argument depends on D being fixed: Proposition 1 rests on the grammar and the kernel

quotient, while the others range over P and S, which now include the new content. Ingestion is itself a *create* step, so it is subject to $validate_S$ (Theorem 1): a schema whose alethic constraints the current P violates is rejected, so a migration stages as derivations or deontic rules until P complies. □

This is what makes a tenant's own schema (Section 3) safe, and it is the first of the two recursions. The second is the boundary at which the guarantees stop.

Definition 9 (Registered function)**.** A definition in *DEFS* is a tuple $\langle name, dom, cod, origin, impl \rangle$ with $origin \in \{compiled, registered\}$. A *compiled* definition has $impl = \rho(o)$ for a compiled object o, total and decidable over finite P. A *registered* definition has *impl* supplied by the host runtime and is in general partial.

Corollary 5 (Enumerable boundary)**.** *The informal surface of the system is the restriction*

$$Filter(eq \circ [s_{origin}, \overline{registered}]) : DEFS, \tag{5}$$

a ρ-application (Proposition 3), and it coincides with the decidability frontier.

Proof. By Definition 9 a compiled definition is finite reduction over a finite population, so it is decidable and total, whereas a registered definition is arbitrary host code whose termination is in general undecidable. The restriction (5) selects exactly the registered definitions, which is exactly the line at which Turing-complete computation re-enters. □

A noun backed by an external system sits on this line: its population is fetched by a registered function, and its facts enter under the open-world assumption [4]. A deontic constraint over such facts is sound but not complete: a reported violation is genuine, while under the open world the absence of one guarantees nothing. Informality is not removed but localized to a queryable fact set, and the trusted base (the parser, the compiler, the verbalizer, and the registered functions) is precisely what (5) returns.

A registered definition that shadows a compiled definition of the same name, and is observationally equal to it, contributes no informality: its meaning is the compiled definition's, decidable by Definition 9, and the shadow may be dropped without changing any value. Eq. (5) is therefore read over the unshadowed registered definitions, so an optimizing runtime does not widen the boundary. The licence for such optimization is Backus's algebra of programs [1, Sec. 12.2]: the laws are equations between the forms the compiled objects represent, so an object may be transformed, ahead of time or at registration, into any equivalent of itself.

5 Conclusion

The gap between domain knowledge and running software, where requirements drift and production breaks, is usually treated as inevitable. It is avoidable. An application is a finite mathematical object: Backus's state machine over Codd's relations and ORM's facts, with every value a ρ-application over P (Proposition 3). Software opens the gap only by pretending to be something other than the object it already is. Stating it plainly closes the gap, because the schema is the database, the readings are the executable, the links are the population, and the workflow is the transition, and none of them was ever separate. Fielding's remaining constraints [10] follow: statelessness because each step is a pure transition on D, cacheability because a representation is a function of P, layering because cells are location-independent, and code-on-demand because the readings that extend the system are themselves data. The operational surface maps into the object the same way: versioning is the event stream, migration is ingestion staged by Corollary 4, and auditability is the derivation chain, recorded and answerable on demand. What remains outside the object is enumerable (Corollary 5), and what is inside is finite (Lemma 1), so it is decidable, and so it is, for once, something a machine can be trusted to have gotten right.

Acknowledgments

Claude (Anthropic) assisted with implementation and manuscript preparation. The architectural decisions and the composition of FFP/AST with REST are the author's.

References

[1] J. Backus, "Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs," *Commun. ACM*, 21(8):613–641, 1978.

[2] E.F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Commun. ACM*, 13(6):377–387, 1970.

[3] E.F. Codd, "Relational Completeness of Data Base Sublanguages," in *Data Base Systems*, Courant Computer Science Symposia 6, Prentice-Hall, 1972, pp. 65–98.

[4] T. Halpin and T. Morgan, *Information Modeling and Relational Databases*, 2nd ed., Morgan Kaufmann, 2008.

[5] T. Halpin and J.P. Wijbenga, "FORML 2," in *Enterprise, Business-Process and Information Systems Modeling (EMMSAD)*, LNBIP 50, pp. 247–260, 2010.

[6] T. Halpin and M. Curland, "Automated Verbalization for ORM 2," in *OTM Workshops*, pp. 1181–1190, 2006.

[7] M.H. van Emden and R.A. Kowalski, "The Semantics of Predicate Logic as a Programming Language," *J. ACM*, 23(4):733–742, 1976.

[8] A. Tarski, "A lattice-theoretical fixpoint theorem and its applications," *Pacific J. Math.*, 5(2):285–309, 1955.

[9] S.C. Kleene, *Introduction to Metamathematics*, North-Holland, 1952.

[10] R.T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.

[11] I. Zužak, I. Budiselić, and G. Delač, "Formal Modeling of RESTful Systems Using Finite-State Machines," in *Proc. ICWE*, LNCS 6757, pp. 346–360, 2011.

[12] S. Gilbert and N. Lynch, "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services," *ACM SIGACT News*, 33(2):51–59, 2002.

[13] T. Halpin and M. Curland, NORMA (Natural Object-Role Modeling Architect), open-source reference implementation, `https://github.com/ormsolutions/NORMA`.

[14] S. Lippert, AREST, open-source reference implementation, `https://github.com/graphdl/arest`.